Diffusion-Based Generation of Novel Pokémon

with Conditional Evolution Modeling

MSML 612

Aaron Cyril John, Yugaank Kalia, Varen Maniktala

Outline

1. Problem Motivation
2. Dataset Overview
3. Data Sources & Curation
4. Preprocessing Pipeline
5. System Architecture
6. Training Procedure
7. Evaluation Plan & Metrics
8. Current Status & Next Steps

Problem Motivation

Goal: Generate novel, visually coherent Pokémon designs under structured conditional controls.

Challenge	Why It Matters
Multi-source style alignment	Official art, game sprites, and 3D renders must correspond
Attribute conditioning	Type, evolution stage, and style must guide generation
Evolution chain coherence	Generated forms must remain visually related across a chain
Domain-specific structure	Pokémon generation is not just image synthesis: it is structured conditional generation

> Standard unconditional image generators cannot capture evolution progression or type-based stylistic cues.

Dataset Overview

Three complementary image sources collected and aligned:

Source	Files	Folders	Role
SugimoriSprites	1,848	1,025	High-quality 2D reference art
PokeSprite	2,853	905	Target game-style sprite output
ProjectPokemon 3D	-	905	Alternate visual domain for cross-format alignment

Mappings generated:

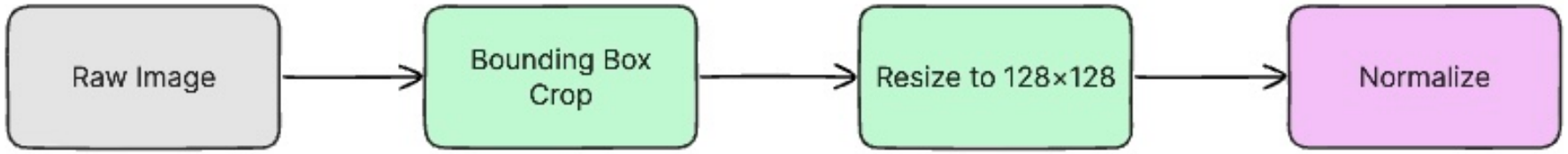
Mapping File	Pairs	Pokémon Folders	Status
`mapping_sugimori_to_sprite.csv`	1,710	905	Auto-generated
`edited_mapping_sugimori_to_sprite.csv`	977	830	Manually verified ✓
`mapping_3d_to_sprite.csv`	2,676	905	Auto-generated

Data Sources: Visual Comparison

Style	Bulbasaur	Charizard
SugimoriSprites (Official Art)		
PokeSprite (Game Sprite)		
ProjectPokemon 3D		

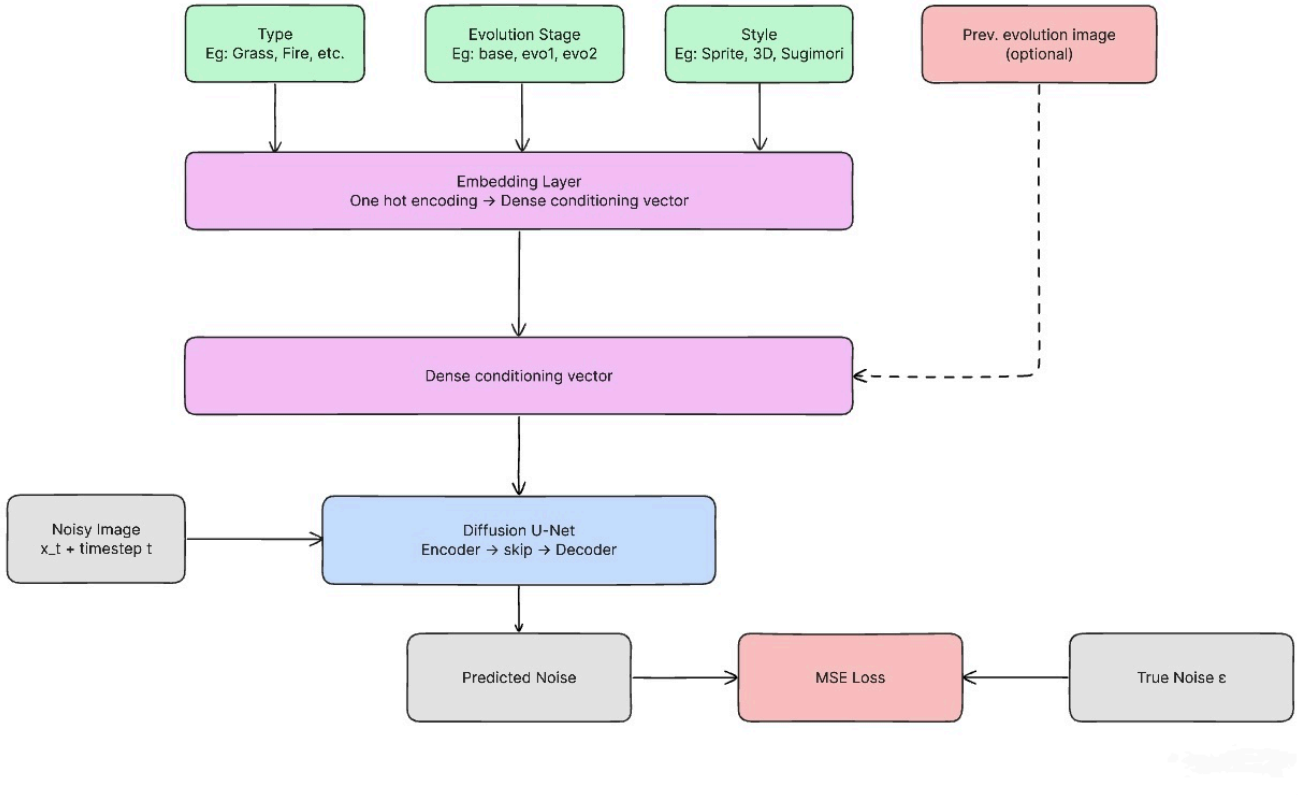
> Each source captures a different visual style of the **same** Pokémon: cross-format alignment is critical.

Preprocessing Pipeline



> Filename normalization → form-variant handling (Mega, Gigantamax) → CSV mapping generation → manual verification via contact-sheet review

System Architecture



Key components:

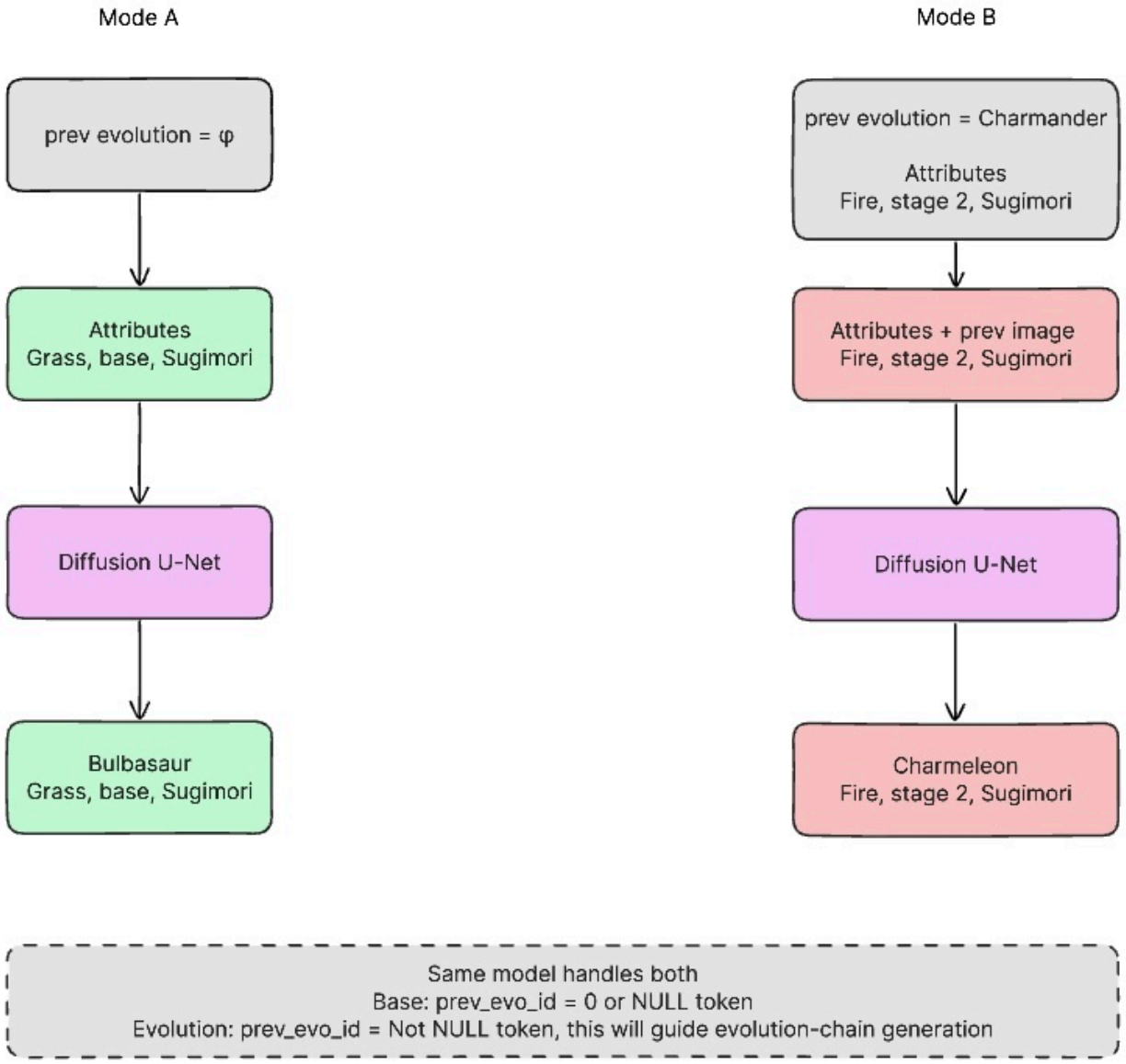
- **Diffusion U-Net backbone**: iterative denoising for image synthesis
- **Attribute conditioning**: type embedding, evolution stage, and style vector
- **Reference-image conditioning path**: guide evolution-chain generation
- **Sprite-focused output space**: final images match game-style visual format

Architecture: Conditioning Design

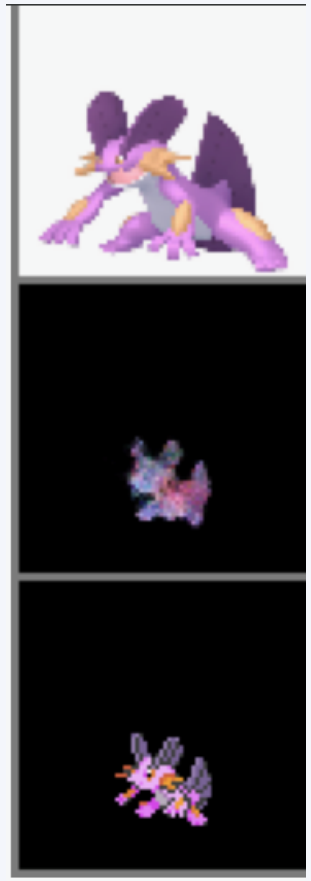
Conditioning Input	Encoding	Purpose
Pokémon Type (e.g., Fire, Water)	One-hot / embedding	Style and color palette bias
Evolution Stage (1 / 2 / 3)	Scalar embedding	Structural complexity control
Visual Style	Domain label embedding	Sprite vs. official art target
Reference Image (optional)	CNN feature extraction	Evolution-chain coherence

> All condition vectors are concatenated and injected into the U-Net via cross-attention at multiple resolutions.

Training Procedure



Preliminary GAN Results

Arceus	Pachirisu	Naganadel
		

Top: Input artwork

Middle: GAN-predicted sprite

Bottom: Ground-truth sprite

GAN captures color palettes and rough silhouettes but lacks fine detail and sharpness compared with the ground-truth sprites.

A diffusion transformer should improve global consistency and recover sharper fine-grained detail relative to this baseline.

Evaluation Plan

Metric	Description	What It Measures
FID (Fréchet Inception Distance)	Distribution distance between generated and real images	Realism & diversity
SSIM	Structural similarity for paired comparisons	Pixel-level fidelity
Diversity Score	Pairwise distance across generated samples	Mode coverage
Qualitative Review	Side-by-side visual inspection	Attribute adherence

Planned ablations (If time allows):

Experiment	Variable
Unconditional vs. attribute-conditioned	Effect of conditioning on quality
With vs. without evolution-reference input	Evolution coherence
Multiple target resolutions	Resolution vs. fidelity trade-off

Current Status

Component	Status
SugimoriSprites collection	Complete (1,848 images)
PokeSprite collection	Complete (2,853 images)
Auto-generated mappings	Complete (4,386 total pairs)
Manual curation & verification	977 verified pairs across 830 Pokémon
Preprocessing & normalization scripts	Complete
Diffusion model architecture design	Finalized
Training pipeline	In progress
Baseline training run	Pending
Quantitative evaluation	Pending

Next Steps

1. **Finalize training pipeline** - convert curated dataset into DataLoader-ready format
2. **Run baseline experiment** - unconditional diffusion model on sprite images
3. **Add conditioning** - integrate type, stage, and style embeddings
4. **Evaluate** - compute FID, SSIM, and diversity scores on held-out Pokémon
5. **Ablation study** - measure impact of each conditioning component
6. **Visual summary** - generate contact sheets of novel Pokémon designs

References

[1] Ho, J., Jain, A., and Abbeel, P. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020. https://arxiv.org/abs/2006.11239

[2] Ronneberger, O., Fischer, P., and Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015. https://arxiv.org/abs/1505.04597

[3] Saharia, C., Chan, W., Saxena, S., et al. *Palette: Image-to-Image Diffusion Models*. SIGGRAPH, 2022. https://arxiv.org/abs/2111.05826

[4] Hugging Face. *Diffusers: State-of-the-art diffusion models for image and audio generation in PyTorch*. https://github.com/huggingface/diffusers

[5] PokéAPI. https://pokeapi.co/

[6] msikma. *pokesprite*. https://github.com/msikma/pokesprite

[7] Project Pokémon. Sprite and asset resources. https://projectpokemon.org/

[8] Pokémon Database. National Pokédex and image resources. https://pokemondb.net/pokedex/national